Kafka Stream: Window

Saturday, 27 June 2026 4:56 PM

What's the use of WINDOW in Kafka Stream:

Kafka streams are infinite. New events keep on arriving. So there is no start, no end.

So, what if our business requirement is to answer: How many orders we received in last 1hr?

Suppose Incoming order events for same key "Ord-1":

10:00 Order-1
10:10 Order-2
10:30 Order-3
11:15 Order-4
11:25 Order-5

Suppose we are grouping without window:

```
orders
.groupByKey()
.count();
```

→

State Store (KTable)

key	Count
Ord-1	5

The KTable only knows:
Orders seen so far = 5.

It does not remember time. Or group orders based on time window.

So, if anyone asks How many orders received between 10:00 - 11:00?

KTable currently can not answer that.

So,

A window is a time-based boundary used to group records from an infinite stream into finite chunks.

Suppose Incoming order events for same key "Ord-1":

10:00 Order-1
10:10 Order-2
10:30 Order-3
11:15 Order-4
11:25 Order-5

Suppose we are grouping with window:

```
orders
.groupByKey()
.windowedBy(...)
.count();
```

→

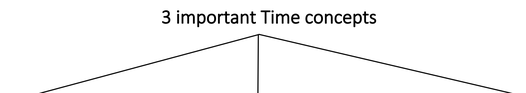
State Store (KTable)

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:00	11:00	3
Ord-1	11:00	12:00	2

Now, if anyone asks How many orders received between 10:00 - 11:00?

Simply Lookup the appropriate window help us to answer that.

Now lets go 1 level more deep into Window:



Event Time

When the event actually happened in the real world.

```
Record:
Key: "o1"

Value: {
  "orderId": "o1",
  "userId": "u1",
  "amount": 500,
  "createdAt": "2026-07-31T10:01:00Z"
}
```

So, "*createdAt*" represents the event time.
Even though Kafka receive the event at 10:04,
Event Time remains "2026-07-31T10:01:00Z"

Kafka Record Timestamp

Every Kafka record has a timestamp.

By default, the producer assigns the timestamp before sending the record.

If the topic is configured to use LogAppendTime, the broker assigns the timestamp when the record is appended to the log.

```
Record:
Key: "o1"

Value: {
  "orderId": "o1",
  "userId": "u1",
  "amount": 500,
  "createdAt": "2026-07-31T10:01:00Z"
}

Kafka Metadata:
Timestamp = 2026-07-31T10:04:12Z
```

By-default in Kafka Stream WINDOW, this record timestamp is used.

Stream Time

Stream time is the highest record timestamp observed so far.

```
Events:
10:00  Order-1  -> Stream time: 10:00
10:10  Order-2  -> Stream time: 10:10
11:00  Order-3  -> Stream time: 11:00
10:30  Order-4  -> Stream time: 11:00
11:40  Order-5  -> Stream time: 11:40
```

Late events can arrive out of order.

11:00 event → arrives first
10:30 event → arrives later

Stream Time advances to 11:00 after processing the first event.

When the 10:30 event arrives later,
Stream Time stays at 11:00.

Before we proceed further, one doubt might be coming:

By-default in Kafka Stream WINDOW record timestamp is used. What if I want Kafka Stream WINDOW to use Event time instead of Record timestamp?

We can easily do that,

By providing custom implemenation of TimestampExtractor interface

Interface already present in Kafka Stream framework

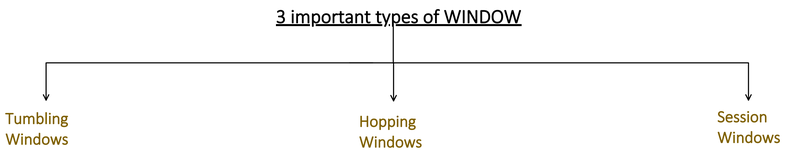
```
public interface TimestampExtractor {
    long extract(ConsumerRecord<Object, Object> record, long partitionTime);
}
```

```
public class OrderTimestampExtractor implements TimestampExtractor {

  @Override
   public long extract(ConsumerRecord<Object, Object> record, long partitionTime) {
         Order order = (Order) record.value();
         return order.getCreatedAt().to Instant().toEpochMilli();
    }
}
```

application.properties

```
spring.kafka.streams.properties.default.timestamp.extractor=com.example.stream.OrderTimestampExtractor
```



1. Tumbling Windows

- Fixed size
- Each record belongs to exactly 1 window.

Example:
Window size = 10mins

Window	Time Range
W1	10:00 to 10:10
W2	10:10 to 10:20
W3	10:20 to 10:30

So event with record timestamp: 10:07 belongs to W1 only.

Below is an example of how data is stored in KTable.

key	Window Start (Inclusive)	Window End (Exclusive)	value
Key-1	10:00	10:05	
Key-1	10:05	10:10	
Key-1	10:10	10:15	
	...	...	
Key-2	10:00	10:05	
Key-2	10:05	10:10	
Key-2	10:10	10:15	
	...	...	

Suppose:
Order Event with "Key-1" arrives at

- 10:02
- 10:13

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Key-1	10:00	10:05	1
Key-1	10:10	10:15	1

There is **no entry** for the 10:05–10:10 window because **no records belonged to that window**.

Use-Case:

- Order per 5 mins
- Revenue per hour
- Api request per minute

etc.

Doubt:
Does window remains open? What I mean say, current time is 11:00 but one delayed event with record timestamp "10:32" comes, will the particular window [10:30-10:35) will be updated?

Without Grace period

As soon as Window is closed, late events are dropped.

Stream Time >= window end time
Consider window as closed

Example:

Window size = 5 mins

Arrival Order	Record Timestamp
1	10:31
2	10:32
3	10:41
4	10:33 (Late)

Ord-1 at record timestamp 10:31
Stream time: 10:31
current time window open: [10:30-10:35)

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:30	10:35	1

Ord-1 at record timestamp 10:32
Stream time: 10:32
current time window open: [10:30-10:35)

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:30	10:35	2

Ord-1 at record timestamp 10:41
Stream time: 10:41
current time window open: [10:40-10:45)
old window before stream time are closed now.

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:30	10:35	2	Window Closed
Ord-1	10:40	10:45	1	

Ord-1 at record timestamp 10:33 (Late)
Stream time: 10:41
current time window open: [10:40-10:45)

Event with record timestamp 10:33 belongs to window
[10:30 - 10:35) which is closed (as Stream time >= 10:35)

So this record will be discarded.

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:30	10:35	2	Window Closed
Ord-1	10:40	10:45	1	

With Grace period

Window is not closed suddenly. We can add Grace period.

Stream Time >= window end time + grace time
after that only consider window as closed

Example:

Window size = 5 mins
Grace time = 10 mins

Arrival Order	Record Timestamp
1	10:31
2	10:32
3	10:41
4	10:33 (Late)

Ord-1 at record timestamp 10:31
Stream time: 10:31
current time window: [10:30-10:35)
with Grace this window will remain open till: 10:45 (10:35+10)

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:30	10:35	1

Ord-1 at record timestamp 10:32
Stream time: 10:32
current time window: [10:30-10:35)
with Grace this window will remain open till: 10:45 (10:35+10)

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:30	10:35	2

Ord-1 at record timestamp 10:41
Stream time: 10:41
current time window: [10:40-10:45)

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:30	10:35	2	Will remain open till 10:45 (10:35 + 10 grace)
Ord-1	10:40	10:45	1	

Ord-1 at record timestamp 10:33 (Late)
Stream time: 10:41
current time window: [10:40-10:45)

Event with record timestamp 10:33 belongs to window
[10:30 - 10:35) which is open till 10:45 (i.e 10:35 + grace time)
So, event is updated in KTable.

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:30	10:35	3
Ord-1	10:40	10:45	1

Method Definitions

No grace period:

TimeWindows.ofSizeWithNoGrace(Duration size)

With grace period:

TimeWindows.ofSizeAndGrace(Duration size, Duration afterWindowEnd)

```
KStream<String, Order> orders = builder.stream(
                                                "orders",
                                                Consumed.with(Serdes.String(), orderSerde)
                                        );

KTable<Windowed<String>, Long> counts = orders
                                                .selectKey((orderId, order) -> order.getUserId())
                                                .groupByKey()
                                                .windowedBy(TimeWindows.ofSizeAndGrace(Duration.ofMinutes(5), Duration.ofMinutes(1)))
                                                .count(Materialized.as("order-count-store"));
```

Or

Key type

```
KTable<Windowed<String>, Long> counts = orders
                                        .selectKey((orderId, order) -> order.getUserId())
                                        .groupByKey()
                                        .windowedBy(TimeWindows.ofSizeAndGrace(Duration.ofMinutes(5), Duration.ofMinutes(1)))
                                        .count(
                                            Materialized.<String, Long, WindowStore<Bytes, byte[]>>as("order-count-store")
                                            .withKeySerde(Serdes.String())
                                            .withValueSerde(Serdes.Long())
                                        );
```

optional

2. Hopping Windows

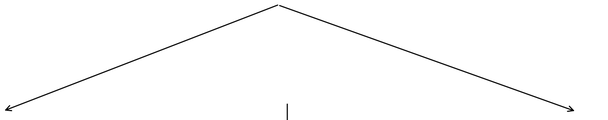
- Fixed size and advanced by fixed interval
- Each record may belongs to multiple windows.

Example:
Window size = 10 mins
Advance by = 5 mins

Window	Time Range
W1	10:00 to 10:10
W2	10:05 to 10:15
W3	10:10 to 10:20

So event with record timestamp: 10:07 belongs to W1 and W2.

Use-Case:
- Order in last 1hr but updated every x mins.
- Moving traffic count
- Rolling revenue calculation
  etc.



Without Grace

With Grace

Example:

Window size = 10 mins

Example:

Window size = 10 mins
Advance (Hop) = 5 mins

Advance (Hop) = 5 mins

Arrival Order	Record Timestamp
1	10:27
2	10:28
3	10:41
4	10:29 (Late)

Windows:
[10:20 - 10:30)
[10:25 - 10:35)
[10:30 - 10:40)
[10:35 - 10:45)
[10:40 - 10:50)

Ord-1 at record timestamp 10:27
Stream time: 10:27
current active windows:
[10:20-10:30)
[10:25-10:35)

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:20	10:30	1
Ord-1	10:25	10:35	1

Ord-1 at record timestamp 10:28
Stream time: 10:28
still belongs to both windows

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:20	10:30	2
Ord-1	10:25	10:35	2

Ord-1 at record timestamp 10:41
Stream time: 10:41
current active window:
[10:35 - 10:45)
[10:40-10:50)

Closed windows:
stream time >= 10:30 so [10:20-10:30) is closed
Similarly
stream time >= 10:35 so [10:25-10:35) is closed

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:20	10:30	2	closed
Ord-1	10:25	10:35	2	closed
Ord-1	10:35	10:45	1	
Ord-1	10:40	10:50	1	

Ord-1 at record timestamp 10:29 (Late)
Stream time: 10:41

Event with record timestamp 10:29 belongs to window
[10:20-10:30)
[10:25-10:35)
and both the windows are closed, so this record will be discarded.

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:20	10:30	2	closed
Ord-1	10:25	10:35	2	closed
Ord-1	10:35	10:45	1	
Ord-1	10:40	10:50	1	

Grace = 10 mins

Arrival Order	Record Timestamp
1	10:27
2	10:28
3	10:41
4	10:29 (Late)

Windows:
[10:20 - 10:30) = closing time: 10:40 (window end +grace)
[10:25 - 10:35) = closing time: 10:45
[10:30 - 10:40) = closing time: 10:50
[10:35 - 10:45) = closing time: 10:55
[10:40 - 10:50) = closing time: 11:00

Ord-1 at record timestamp 10:27
Stream time: 10:27
current active windows:
[10:20-10:30)
[10:25-10:35)

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:20	10:30	1
Ord-1	10:25	10:35	1

Ord-1 at record timestamp 10:28
Stream time: 10:28
still belongs to both windows

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:20	10:30	2
Ord-1	10:25	10:35	2

Ord-1 at record timestamp 10:41
Stream time: 10:41
current active window:
[10:25 - 10:35) -> 10:35 + 10(grace) = 10 -> till 10:45
[10:35 - 10:45)
[10:40-10:50)

Closed windows:
stream time >= 10:30 + 10(grace)
so [10:20-10:30) is closed

Similarly
stream time >= 10:35 so [10:25-10:35) is closed

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:20	10:30	2	closed
Ord-1	10:25	10:35	2	Open till 10:45
Ord-1	10:35	10:45	1	
Ord-1	10:40	10:50	1	

Ord-1 at record timestamp 10:29 (Late)
Stream time: 10:41

Event with record timestamp 10:29 belongs to window
[10:20-10:30) - closed
[10:25-10:35)
so this record will be accepted and updated
in open window only.

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:20	10:30	2	closed
Ord-1	10:25	10:35	3	
Ord-1	10:35	10:45	1	
Ord-1	10:40	10:50	1	

Method Definitions

No grace period:

```
TimeWindows.ofSizeWithNoGrace(Duration size)
            .advanceBy(Duration advance)
```

With grace period:

```
TimeWindows.ofSizeAndGrace(Duration size,
Duration afterWindowEnd)
            .advanceBy(Duration advance)
```

```
KStream<String, Order> orders = builder.stream(

                                    "orders",

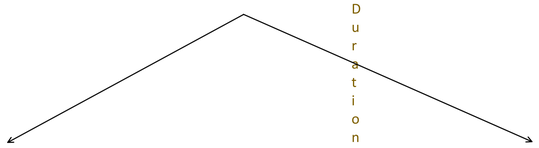
                                    Consumed
                                    .with(Serd
                                    es.String(),
                                    orderSerd
                                    e)
                            );

KTable<Windowed<String>, Long> counts = orders
                                    .selectKey(
                                    (orderId,
                                    order) ->
                                    order.getU
                                    serId())
                                     .groupByK
                                    ey()
                                     .windowe
                                    dBy(Time
                                    Windows.
                                    ofSizeAnd
                                    Grace(Dur
                                    ation.ofMi
                                    nutes(5),
                                    Duration.o
                                    fMinutes(1
                                    ))
                                    .advanceBy(Duration.ofMinutes(1))
                                    )
                                    .count(Ma
                                    terialized.a
                                    s("order-
                                    count-
                                    store"));
```

Session Windows

Group events that occurs close together in time
A new session (group) is created after a period of inactivity.
There is no window size.

Example:
Inactivity Gap = 5 mins



Without Grace:

Example:

Inactivity Gap = 5 mins

Arrival Order	Record Timestamp
1	10:27
2	10:29
3	10:41
4	10:32 (Late)

Ord-1 at record timestamp 10:27
Stream time: 10:27
No existing session

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:27	10:27	1

Ord-1 at record timestamp 10:29

With Grace:

Example:

Inactivity Gap = 5 mins
Grace = 14mins

Arrival Order	Record Timestamp
1	10:27
2	10:29
3	10:41
4	10:32 (Late)

Ord-1 at record timestamp 10:27
Stream time: 10:27
No existing session

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:27	10:27	1

Ord-1 at record timestamp 10:29

Stream time: 10:29
Difference:
10:29-10:27 = 2mins
since 2mins < 5mins
It belongs to same session

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:27	10:29	2

Ord-1 at record timestamp 10:41
Stream time: 10:41
Difference:
10:41-10:29 = 12minutes
12minutes > 5minutes, so new session will be created.
Also
Stream time > session end time (10:41 > 10:29), so previous session will be closed.

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:27	10:29	2	closed
Ord-1	10:41	10:41	1	

Ord-1 at record timestamp 10:32 (Late)
Stream time: 10:41
1.)10:32- 10:29 = 3minutes < 5minutes
So it belongs to 1st session, But that's already Closed.

2.)10:41-10:32 = 9minutes
not withing the inactivity gap (9minutes > 5minutes)
so can not be added in this too.

So this event will be discarded.

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:27	10:29	2	closed
Ord-1	10:41	10:41	1	

Stream time: 10:29
Difference:
10:29-10:27 = 2mins
since 2mins < 5mins
It belongs to same session

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:27	10:29	2

Ord-1 at record timestamp 10:41
Stream time: 10:41
Difference:
10:41-10:29 = 12minutes
12minutes > 5minutes, so new session will be created.
Also
close all previous session when
Stream time > session end time + grace

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:27	10:29	2	Open till 10:43 (10:29+14min)
Ord-1	10:41	10:41	1	

Ord-1 at record timestamp 10:32 (Late)
Stream time: 10:41
1.)
10:32- 10:29 = 3minutes < 5minutes
So it belongs to 1st session, and its still open
So this event will be accepted.

2.)
10:41-10:32 = 9minutes > 5 minutes
so these 2 session can not be merged.

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:27	10:32	3	Open till 10:46 (10:32+14)
Ord-1	10:41	10:41	1	
	Interesting use-case of merging session:			

Example:

Inactivity Gap = 5 mins
Grace = 14mins

Arrival Order	Record Timestamp
1	10:27
2	10:29
3	10:36
4	10:32 (Late)

Ord-1 at record timestamp 10:27
Stream time: 10:27
No existing session

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:27	10:27	1

Ord-1 at record timestamp 10:29
Stream time: 10:29
Difference:
10:29-10:27 = 2mins
since 2mins < 5mins
It belongs to same session

key	Window Start (Inclusive)	Window End (Exclusive)	Count
Ord-1	10:27	10:29	2

Ord-1 at record timestamp 10:36
Stream time: 10:36
Difference:
10:36-10:29 = 7 minutes
7minutes > 5minutes, so new session will be created.
Also
close all previous session when
Stream time > session end time + grace

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:27	10:29	2	Open till 10:43 (10:29+14min)
Ord-1	10:36	10:36	1	

Ord-1 at record timestamp 10:32 (Late)
Stream time: 10:36
1)
10:32- 10:29 = 3minutes < 5minutes
So it belongs to 1st session

2)
10:36- 10:32 = 4minutes < 5minutes
So it belongs to 2nd session

So this late event connect both sessions, so kafka stream will merge the session

key	Window Start (Inclusive)	Window End (Exclusive)	Count	
Ord-1	10:27	10:36	4	Open till 10:50 (10:36+14)

Use-cases:
Login session (how long user browse, also we can compute average session duration etc.)

Method Definitions

No grace period:

SessionWindows.ofInactivityGapWithNoGrace(Duration inactivityGap)

With grace period:

SessionWindows.ofInactivityGapAndGrace(Duration inactivityGap, Duration afterWindowEnd)

```
KStream<String, Order> orders = builder.stream(
                                                "orders",
                                                Consumed
                                                .with(Serdes.String(),
                                                orderSerde)
                                        );

KTable<Windowed<String>, Long> session = orders
                                                .selectKey(
                                                (orderId,
                                                order) ->
```

```
order.getU
serId())
  .groupByK
ey()
  .windowe
dBy(Sessio
nWindows
.ofInactivit
yGapAndG
race(Durat
ion.ofMin
utes(10),

                                                    D
                                                    u
                                                    r
                                                    a
                                                    t
                                                    i
                                                    o
                                                    n
                                                    .
                                                    o
                                                    f
                                                    M
                                                    i
                                                    n
                                                    u
                                                    t
                                                    e
                                                    s
                                                    (
                                                    2
                                                    )
                                                    )
                                                    )

  .count(Ma
terialized.a
s("store-
name"));
```